

Dummy 手眼标定笔记

- 1.机械臂坐标、视觉/摄像头坐标、像素坐标之间的转换关系
- 2.两种方式，眼在手上、眼在手外
- 3.标定版，类二维码，ArUco
(此教程已保存录屏)



上面视频方案：

《ROS 下 UR+Realsense 相机进行手眼标定_realsense 相机坐标系 ros-CSDN 博客.pdf》

https://blog.csdn.net/weixin_42157689/article/details/134719924

git clone https://github.com/pal-robotics/aruco_ros.git

git clone https://github.com/IFL-CAMP/easy_handeye.git

git clone https://github.com/lagadic/vision_visp.git

git clone https://github.com/marcoesposito1988/easy_handeye2.git

(后备: https://gitlab.com/raymondchaneee/aruco_ros2)

关于 ARUCO 是什么?	
ARUCO_ros 是 PAL Robotics 的一个 ROS 包，用于跟踪和定位 ARUCO marker (快速检测算法生成，如下图)。	
主要功能表	
功能	说明
检测 ARUCO markers	从相机图像检测到一个或多个 ARUCO 标记
检测 3D 位置	计算每个标记相对于相机的 3D 位置 (位置-速度)
发布 ROS 话题	发布话题: <code>aruco_msgs/aruco_tracking</code> ，发布所有标记的位置
发布 TF 广播	发布在 ROS 发布 ARUCO 的 TF 变换关系

用法: `ARUCO_ros` 主要包含 ARUCO 包和 ARUCO 包，没有 ARUCO 包。

→ 用视觉伺服 (Visual Servoing) 和目标跟踪 (Object Tracking) 的。

vision_visp 项目包含的包名	
包名	说明
vision_visp	视觉伺服 (Visual Servoing) 和目标跟踪 (Object Tracking) 的 ROS 包。
vision_visp_msgs	视觉伺服 (Visual Servoing) 和目标跟踪 (Object Tracking) 的 ROS 消息。
vision_visp_srvs	视觉伺服 (Visual Servoing) 和目标跟踪 (Object Tracking) 的 ROS 服务。
vision_visp_plugins	视觉伺服 (Visual Servoing) 和目标跟踪 (Object Tracking) 的 ROS 插件。
vision_visp_examples	视觉伺服 (Visual Servoing) 和目标跟踪 (Object Tracking) 的 ROS 示例。

安装 vision_visp:

sudo apt-get install libvisp-dev (先安装 ViSP 库)

sudo apt update (找不到源就需要 update)

sudo apt install ros-humble-image-pipeline (上 vpn)

cd /home/hata_ros/apps/vision_visp_ws

colcon build --symlink-install

source install/setup.sh

ros2 launch visp_auto_tracker tutorial_launch.xml

安装 aruco_ros:

```
cd /home/hata_ros/apps/aruco_ros_ws
colcon build
source install/setup.sh
ros2 launch aruco_ros single.launch.py markerId:=26 markerSize:=0.08
eye:="right"
```

安装 easy_handeye2:

更换原来的 easy_handeye (只对应 ROS1)

```
rosdep install -iyr --from-paths src
colcon build
source install/setup.sh
```

启动机械臂

```
ros2 launch dummy_moveit_config demo.launch.py
ros2 launch dummy_moveit_config demo_custom.launch.py
```

启动摄像头 (RGB+DEPTH)

```
ros2 launch realsense2_camera rs_launch.py enable_rgbd:=true
enable_sync:=true align_depth.enable:=true
rgb_camera.color_profile:=1280x720x30
```

启动 aruco

(注意使用兼容的 dictionary, 例如 aruco origin)

(注意一定要可视化一下 /aruco_single/result 的图像, 才会开始发布 camera_marker)

```
ros2 launch aruco_ros single_custom.launch.py \
  camera_info:="/camera/camera/color/camera_info" \
  image:="/camera/camera/color/image_raw" \
  marker_id:=6 \
  marker_size:=0.05 \
  reference_frame:=camera_link \
  camera_frame:=camera_color_optical_frame \
  marker_frame:=camera_marker
```

启动 easy_handeye

```
ros2 launch easy_handeye2 calibrate.launch.py \
  name:=dummy_eob_calib \
  calibration_type:=eye_on_base \
  robot_base_frame:=base_link \
  robot_effector_frame:=link6_1_1 \
  tracking_base_frame:=camera_link \
  tracking_marker_frame:=camera_marker
```

查看相关 topic

```
ros2 topic echo /tf_static
ros2 topic echo /tf
ros2 run image_view image_view image:=/aruco_single/result
ros2 run image_view image_view image:=/camera/camera/color/image_raw
(可以用 rqt 的 image view 来代替)
```

标定

目前版本只要 sample 足够多样本就会自动计算 arm 和 cam 的转换矩阵

eye on base 每个 sample 后的转换矩阵偏差跳动有点厉害

按 save 保存到 /home/hata_ros/.ros2/easy_handeye2/calibrations/
dummy_eob_calib.calib

(log 可以看到)

使用 rviz 增加 tf 的可视化可以查看除了 dummy 以外 camera 的坐标 (但 camera 位置很奇怪??)

发布

用于后续手眼结合场景

ros2 launch easy_handeye2 publish.launch.py name:=dummy_eob_calib

(name 与上面启动标定的时候一致, 成功后会多了一个 base_link 到 camera_link 的转换)

使用 rviz 可以查看 base_link 到 camera_link 的转换

(看出坐标跟实际偏差应该是蛮大的, 后期最蠢就是自己手动调整转换矩阵的结果文件)

```
~Cuda_ronhata: /apps/easy_handeye2_ws/src/easy_handeye2/launch $ ros2 topic echo /tf_static
transforms:
- header:
  stamp:
    sec: 1745977576
    nanosec: 973954255
  frame_id: base_link
  child_frame_id: camera_link
  transform:
    translation:
      x: 0.020056885960580464
      y: -0.17525847032047503
      z: 0.35914597316351426
    rotation:
      x: -0.23712166210765764
      y: 0.16852758237964284
      z: 0.04926739462961478
      w: 0.9568578589475434
...
```

